

Snapshots of volumes cannot be accessed directly, but they can be cloned, backed up, rolled back to, and so on.

4.7 Simplified administration

Most importantly, ZFS provides a greatly simplified administration model. Through the use of hierarchical file system layout, property inheritance, and automanagement of mount points and NFS share semantics, ZFS makes it easy to create and manage file systems without needing multiple commands or editing configuration files. You can easily set quotas or reservations, turn compression on or off, or manage mount points for numerous file systems with a single command. Devices can be examined or repaired without having to understand a separate set of volume manager commands. You can take an unlimited number of instantaneous snapshots of file systems. You can backup and restore individual file systems. ZFS manages file systems through a hierarchy that allows for this simplified management of properties such as quotas, reservations, compression, and mount points. In this model, file systems become the central point of control. File systems themselves are very cheap (equivalent to a new directory), so you are encouraged to create a file system for each user, project, workspace, and so on. This design allows you to define fine-grained management points.

4.8 Getting Started with ZFS

In this section, we will show some simple commands which you can use in order to make use of ZFS file system.

So to make a 1 GB file, you would need to execute `$ mkfile 1g myfile`. To create a 100 MB file you will execute `$ mkfile 100m myfile`.

4.8.1 Creating a Basic ZFS file system

ZFS administration has been designed with simplicity in mind. Among the goals of the ZFS design is to reduce the number of commands needed to create a usable file system. When you create a new pool, a new ZFS file system is created and mounted automatically. The following example illustrates how to create a simple mirrored storage pool named `tank` and a ZFS file system named `tank` in one command. Assume that the whole of `/dev/dsk/c1t0d0` and `/dev/dsk/c2t0d0` are available for use. You can use two files (which are bigger than 128 MB) instead of the disks in order to try this out.

```
# zpool create tank mirror c1t0d0 c2t0d0
```

OpenSolaris and Solaris have a standard naming convention for storage